FDPACK: Finite Difference Package with Adaptive Offset Selection

Roland Tóbiás[1]

[1] *Department of Chemistry, University of Vermont, Burlington, VT, 05405, USA (rtobias@uvm.edu)*

This document presents the algorithms underlying the automatic offset selection in FDPACK, a modern FORTRAN library for numerical differentiation of multivariate functions. The two core routines, get_noise_table and get_fdoffsets, predict the direction-dependent noise floor of the objective function and derive the offsets that minimize the total differentiation error, respectively. Both are designed to reach maximum accuracy with a minimal number of function evaluations. To the best of our knowledge, FDPACK is the only tool capable of optimizing finite-difference offsets for numerical Hessians. The resulting offsets can be passed to get_num_grad and get_num_hess, which evaluate the gradient and Hessian *via* forward/central difference formulas, respectively.

1. Introduction

Numerical differentiation is a fundamental tool in scientific computing, with applications ranging from optimization and sensitivity analysis to the solution of differential equations. The finite difference method approximates derivatives of a multivariate function $f(\mathbf{x})$, where $\mathbf{x} \in \mathbb{R}^n$ denotes the variable vector, by evaluating f at points near $\mathbf{x}$.[1] In this work, only gradient components ($g_i = \partial f/\partial x_i$) and Hessian entries ($h_{ij} = \partial^2 f/\partial x_i \partial x_j$) are considered.

Central to all finite difference schemes is the offset ω, defined here as the quantity appearing in the denominator of the finite difference expression. For forward differences ($p = 1$), the function is perturbed by ω along the relevant direction, whereas the perturbation is $\omega/2$ for central differences ($p = 2$), so that the denominator retains the simple form ω^p without additional constant factors in either case. This convention, adopted throughout FDPACK, ensures that the truncation error scales as $t\,\omega^p$ regardless of the scheme, where t is a suitable constant.

The accuracy of a finite difference approximation depends critically on the choice of ω: (a) if ω is too large, it introduces truncation error from the neglected higher-order terms in the Taylor expansion, otherwise (b) it amplifies roundoff and any inherent noise in the function evaluation.[2] The optimal offset lies at the balance point between the two competing error sources, and its value depends on both the local behavior of the function and the noise level of its evaluations. FDPACK addresses this challenge through a systematic, adaptive procedure that first estimates the function noise in each coordinate direction, then determines the offset that minimizes the total error for each entry of the gradient vector and Hessian matrix.[3]

2. Theoretical background

2.1. Finite difference stencils

The FDPACK library supports both forward ($p = 1$) and central ($p = 2$) difference schemes for the gradient and Hessian. The gradient component g_i is approximated as

$$g_i = \frac{1}{\omega_i} \sum\nolimits_k w_k\, f(\mathbf{x} + a_k \omega_i \mathbf{e}_i), \tag{1}$$

in which $\mathbf{e}_i$ is the ith unit vector, ω_i is the offset in direction $\mathbf{e}_i$, and the weights w_k and node shifts a_k depend on the scheme. An arbitrary Hessian entry h_{ij} is approximated as

$$h_{ij} = \frac{1}{\omega_{ij}^{[1]}\omega_{ij}^{[2]}} \sum\nolimits_k w_k^{L_{ij}}\, f(\mathbf{x} + a_k^{L_{ij}} \omega_{ij}^{[1]} \mathbf{e}_i + b_k^{L_{ij}} \omega_{ij}^{[2]} \mathbf{e}_j), \tag{2}$$

where $\omega_{ij}^{[1]}$ and $\omega_{ij}^{[2]}$ are the offsets along the $\mathbf{e}_i$ and $\mathbf{e}_j$ directions, respectively. The superscript L_{ij} designates the stencil type: L_{ij} = pure for diagonal entries ($i = j$) and L_{ij} = mix for off-diagonal entries ($i \neq j$). The norm of the corresponding weight vector is symbolized by $\kappa^{L_{ij}}$, which will play an important role in the treatment of numerical noise (*vide infra*). The stencil coefficients and κ factors are summarized in Table 1.

For mixed entries, both offsets are meaningful, because both directions can be associated with independent perturbations. A diagonal entry, however, must be perturbed with a single offset, due to the coincidence of directions $\mathbf{e}_i$ and $\mathbf{e}_j$. By convention, FDPACK extracts the diagonal Hessian offset from $\omega_{ii} \equiv \omega_{ii}^{[2]}$, while leaving $\omega_{ii}^{[1]}$ available for storing the gradient offset ω_i. This convention allows a single 3D array $\omega_{ij}^{[l=1,2]}$ to hold all necessary offsets. For the Hessian matrix, only its lower-triangular entries is computed and their values are copied into the upper triangular part. **In other words, the current implementation supposes that f is totally differentiable.**

Table 1: Definitions of finite difference stencils used in this study[a]

Derivative	Label	p	w	a	b	κ
Gradient	grad	1	$-1, 1$	$0, 1$	–	$\sqrt{2}$
		2	$-1, 1$	$-0.5, 0.5$	–	$\sqrt{2}$
Hessian	pure	1	$1, -2, 1$	$0, 1, 2$	–	$\sqrt{6}$
		2	$4, -8, 4$	$-1, 0, 1$	–	$4\sqrt{6}$
	mix	1	$1, -1, -1, 1$	$1, 1, 0, 0$	$1, 0, 1, 0$	2
		2	$4, -4, -4, 4$	$0.5, 0.5, -0.5, -0.5$	$0.5, -0.5, 0.5, -0.5$	8

[a] For central difference schemes ($p = 2$), the shifts are halved so that the denominator contains only the ω^p powers in accordance with the offset specification given in Sec. 1. The κ values represent the norms of the weight vectors and they will prove useful in the estimation of computational noise.

2.2. Optimal offsets

The total error in the finite difference approximation of the g_i entry has two contributions: truncation error from the neglected Taylor terms, and function noise amplified by the finite difference denominator:

$$\varepsilon_i \approx t_i \omega_i^p + \kappa^{\text{grad}} f_i^{\text{noise}} / \omega_i. \tag{3}$$

In this formula, (a) t_i designates the truncation coefficient associated with the g_i entry, (b) f_i^{noise} means the function noise amplitude in direction $\mathbf{e}_i$ (it may vary across directions), and (c) κ^{grad} is the stencil noise factor (see Table 1). To find the optimum of the ε_i error, $\mathrm{d}\varepsilon_i/\mathrm{d}\omega_i = 0$ must be sought, from which the optimal offset is

$$\omega_i^{\text{opt}} = \left(\frac{\kappa^{\text{grad}} f_i^{\text{noise}}}{p\, t_i} \right)^{1/(p+1)}. \tag{4}$$

For the h_{ii} component with a truncation coefficient t_{ii}, the noise is amplified by the second power of ω_{ii} in the denominator of Eq. (2). This leads to a total error estimated as

$$\varepsilon_{ii} \approx t_{ii} \omega_{ii}^p + \kappa^{\text{pure}} f_i^{\text{noise}} / \omega_{ii}^2, \tag{5}$$

from which $\mathrm{d}\varepsilon_{ii}/\mathrm{d}\omega_{ii} = 0$ gives

$$\omega_{ii}^{\text{opt}} = \left(\frac{2\kappa^{\text{pure}} f_i^{\text{noise}}}{p\, t_{ii}} \right)^{1/(p+2)}. \tag{6}$$

For an off-diagonal entry h_{ij} ($i \neq j$), $\omega_{ij}^{[1]}$ can be viewed as the gradient offset in direction $\mathbf{e}_i$, while $\omega_{ij}^{[2]}$ controls the perturbation of the $g_i(\mathbf{x})$ function along $\mathbf{e}_j$. As the optimal offset is unlikely to change significantly for g_i at point $\mathbf{x} \pm \omega_{ij}^{[2]} \mathbf{e}_j$, one can adopt

$$\omega_{ij}^{[1]:\text{opt}} = \omega_i^{\text{opt}}, \tag{7}$$

and optimize only $\omega_{ij}^{[2]}$. Then, the total error of this second offset reads

$$\varepsilon_{ij} \approx t_{ij} (\omega_{ij}^{[2]})^p + \kappa^{\text{mix}}\, g_{ij}^{\text{noise}} / \omega_{ij}^{[2]}. \tag{8}$$

For the h_{ij} entry, t_{ij} symbolizes the truncation coefficient and g_{ij}^{noise} is the combined gradient noise defined as

$$g_{ij}^{\text{noise}} = \sqrt{(f_i^{\text{noise}})^2 + (f_j^{\text{noise}})^2} \,/\, \omega_i^{\text{opt}}, \tag{9}$$

assuming uncorrelated noise in directions $\mathbf{e}_i$ and $\mathbf{e}_j$. Therefore, $\mathrm{d}\varepsilon_{ij}/\mathrm{d}\omega_{ij}^{[2]} = 0$ results in

$$\omega_{ij}^{[2]:\text{opt}} = \left(\frac{\kappa^{\text{mix}}\, g_{ij}^{\text{noise}}}{p\, t_{ij}} \right)^{1/(p+1)}. \tag{10}$$

Since the truncation coefficients entering Eqs. (4), (6), and (10) are not known *a priori*, they must be estimated from derivative values evaluated at two distinct offsets (the same holds for the f_i^{noise} values, which can be calculated using the algorithm presented in Sec. 3.2). For these formulas, a raw truncation estimator is obtained through two-point pth-order Richardson extrapolation:[4]

$$t^{\text{raw}} \equiv \mathcal{R}_p(\omega^{\text{loose}}, \omega^{\text{tight}}, D^{\text{loose}}, D^{\text{tight}}) = \frac{|D^{\text{loose}} - D^{\text{tight}}|}{(\omega^{\text{loose}})^p - (\omega^{\text{tight}})^p}, \tag{11}$$

where D^{loose} and D^{tight} denote derivative estimates related to offsets ω^{loose} and ω^{tight}, respectively ($\omega^{\text{loose}} \geq \omega^{\text{tight}}$).

The main issue with Eq. (11) is that t^{raw} may become excessively large or arbitrarily small, producing meaningless offsets in Eqs. (4), (6), and (10). Consequently, the truncation coefficient must be restricted to an admissible range, for which the optimal offset relations can be written in a unified form:

$$\omega = \left(\frac{q\kappa\nu}{pt}\right)^{1/(p+q)}, \tag{12}$$

where t, ν, κ, and q are the truncation coefficient, the noise level, the amplification factor, and the noise exponent associated with the given offset relation, respectively. Inversion of Eq. (12) yields a so-called switch function:

$$\mathcal{S}_{pq}(\kappa, \nu, \omega) = \frac{q\kappa\nu}{p\omega^{p+q}}, \tag{13}$$

which provides the truncation coefficient at a specific ω value. To guarantee that ω falls into an $[\omega^{\min}, \omega^{\max}]$ range, t must remain between $t^{\min} = \mathcal{S}_{pq}(\kappa, \nu, \omega^{\max})$ and $t^{\max} = \mathcal{S}_{pq}(\kappa, \nu, \omega^{\min})$. In this situation, the final form of the t coefficient can be defined as

$$t = \mathcal{C}(t^{\mathrm{raw}}, t^{\min}, t^{\max}), \tag{14}$$

where $\mathcal{C}(v, v_{\min}, v_{\max}) = \max(v_{\min}, \min(v, v_{\max}))$ is the clamp function restricting v to the $[v_{\min}, v_{\max}]$ interval.

The above relations are now specialized to the three derivative classes. For the gradient components, the truncation coefficients are obtained as

$$t_i = \mathcal{C}(T_i^{\mathrm{raw}}, t_i^{\min}, t_i^{\max}) \tag{15}$$

with

$$t_i^{\mathrm{raw}} = \mathcal{R}_p(\omega_i^{\mathrm{loose}}, \omega_i^{\mathrm{tight}}, g_i^{\mathrm{loose}}, g_i^{\mathrm{tight}}), \tag{16}$$

$$t_i^{\min} = \mathcal{S}_{p1}(\kappa^{\mathrm{grad}}, f_i^{\mathrm{noise}}, \omega_i^{\max}), \tag{17}$$

$$t_i^{\max} = \mathcal{S}_{p1}(\kappa^{\mathrm{grad}}, f_i^{\mathrm{noise}}, \omega_i^{\min}), \tag{18}$$

which necessitate the evaluation of the gradient estimates g_i^{loose} and g_i^{tight} at offsets $\omega_i^{\mathrm{loose}}$ and $\omega_i^{\mathrm{tight}}$, respectively. For the (i, i) Hessian entry, the truncation coefficient is

$$t_{ii} = \mathcal{C}(t_{ii}^{\mathrm{raw}}, t_{ii}^{\min}, t_{ii}^{\max}) \tag{19}$$

with

$$t_{ii}^{\mathrm{raw}} = \mathcal{R}_p(\omega_{ii}^{\mathrm{loose}}, \omega_{ii}^{\mathrm{tight}}, h_{ii}^{\mathrm{loose}}, h_{ii}^{\mathrm{tight}}), \tag{20}$$

$$t_{ii}^{\min} = \mathcal{S}_{p2}(\kappa^{\mathrm{pure}}, f_i^{\mathrm{noise}}, \omega_i^{\max}), \tag{21}$$

$$t_{ii}^{\max} = \mathcal{S}_{p2}(\kappa^{\mathrm{pure}}, f_i^{\mathrm{noise}}, \omega_i^{\min}), \tag{22}$$

where the two Hessian-diagonal estimates, h_{ii}^{loose} and h_{ii}^{tight}, are computed at offsets $\omega_{ii}^{\mathrm{loose}}$ and $\omega_{ii}^{\mathrm{tight}}$, respectively. For the (i, j) Hessian entry $(i \neq j)$,

$$t_{ij} = \mathcal{C}(t_{ij}^{\mathrm{raw}}, t_{ij}^{\min}, t_{ij}^{\max}), \tag{23}$$

with

$$t_{ij}^{\mathrm{raw}} = \mathcal{R}_p(\omega_{ij}^{[2]:\mathrm{loose}}, \omega_{ij}^{[2]:\mathrm{tight}}, h_{ij}^{\mathrm{loose}}, h_{ij}^{\mathrm{tight}}), \tag{24}$$

$$t_{ij}^{\min} = \mathcal{S}_{p1}(\kappa^{\mathrm{mix}}, g_{ij}^{\mathrm{noise}}, \omega_j^{\max}), \tag{25}$$

$$t_{ij}^{\max} = \mathcal{S}_{p1}(\kappa^{\mathrm{mix}}, g_{ij}^{\mathrm{noise}}, \omega_j^{\min}), \tag{26}$$

in which the estimated Hessian entries h_{ij}^{tight} and h_{ij}^{loose} correspond to offsets $\omega_{ij}^{[2]:\mathrm{tight}}$ and $\omega_{ij}^{[2]:\mathrm{loose}}$, respectively (both using the offset ω_i^{opt} in direction $\mathbf{e}_i$). Alternatively, t_{ij}^{raw} can be approximated *via* a cheaper gradient-only guess:

$$t_{ij}^{\mathrm{raw}} \approx \min(t_i^{\mathrm{raw}}/\omega_j^{\mathrm{opt}}, t_j^{\mathrm{raw}}/\omega_i^{\mathrm{opt}}), \tag{27}$$

which is a good estimate when the mixed truncation error is not dominated by cross-direction contributions.

3. Implementation details

The algorithms described in Sec. 2 are implemented across two modules. The `mod_fdbase` module contains the finite-difference stencils and the top-level differentiation routines, whereas `mod_fdtune` implements the adaptive offset selection. The two modules are coupled through the `fdconfig` type, which holds the finite-difference scheme and the offset array as a thread-private global state. The key module components are described in the following subsections.

3.1. Stencil-related functions in module **`mod_fdbase`**

The `mod_fdbase` module exposes three top-level functions: `get_num_grad` and `get_num_hess` compute the gradient vector and the Hessian matrix, respectively, whereas `diag_num_hess` additionally performs a spectral decomposition of the Hessian *via* the LAPACK subroutine `DSYEV`[5] (available only if the `USE_LAPACK` preprocessor flag is set at compile time). All three routines delegate the actual stencil evaluations to four low-level functions (namely, `cdgrad`, `fwgrad`, `cdhess`, and `fwhess`) which implement the finite-difference formulas described below.

For both schemes, the two gradient routines take advantage of Eq. (1). Relying on a single forward perturbation, `fwgrad` approximates g_i at $\mathbf{x}$ as

$$g_i = \frac{f(\mathbf{x} + \omega_i \mathbf{e}_i) - f(\mathbf{x})}{\omega_i}. \tag{28}$$

By contrast, `cdgrad` samples the function symmetrically around $\mathbf{x}$, yielding a central difference approximation,

$$g_i = \frac{f(\mathbf{x} + \frac{1}{2}\omega_i \mathbf{e}_i) - f(\mathbf{x} - \frac{1}{2}\omega_i \mathbf{e}_i)}{\omega_i}, \tag{29}$$

where the half-step convention ensures consistency with the offset specification of Sec. 2.1. Similarly, the two Hessian routines take advantage of Eq. (2) for both schemes. For diagonal entries, `cdhess` applies the three-point formula

$$h_{ii} = \frac{4f(\mathbf{x} - \frac{1}{2}\omega_{ii}^{[2]}\mathbf{e}_i) - 8f(\mathbf{x}) + 4f(\mathbf{x} + \frac{1}{2}\omega_{ii}^{[2]}\mathbf{e}_i)}{(\omega_{ii}^{[2]})^2}, \tag{30}$$

while off-diagonal entries ($i \neq j$) follow from a four-point formula,

$$\begin{aligned} h_{ij} = \frac{1}{\omega_{ij}^{[1]}\omega_{ij}^{[2]}} \Big[& 4f(\mathbf{x} + \tfrac{1}{2}\omega_{ij}^{[1]}\mathbf{e}_i + \tfrac{1}{2}\omega_{ij}^{[2]}\mathbf{e}_j) - 4f(\mathbf{x} + \tfrac{1}{2}\omega_{ij}^{[1]}\mathbf{e}_i - \tfrac{1}{2}\omega_{ij}^{[2]}\mathbf{e}_j) \\ & -4f(\mathbf{x} - \tfrac{1}{2}\omega_{ij}^{[1]}\mathbf{e}_i + \tfrac{1}{2}\omega_{ij}^{[2]}\mathbf{e}_j) + 4f(\mathbf{x} - \tfrac{1}{2}\omega_{ij}^{[1]}\mathbf{e}_i - \tfrac{1}{2}\omega_{ij}^{[2]}\mathbf{e}_j) \Big]. \end{aligned} \tag{31}$$

To handle the forward case, `fwhess` cancels the first-order contributions from the double perturbation using $\mathbf{g}$ at $\mathbf{x}$, and all entries (both $i = j$ and $i \neq j$) are obtained from a unified formula,

$$h_{ij} = \frac{f(\mathbf{x} + \omega_{ij}^{[1]}\mathbf{e}_i + \omega_{ij}^{[2]}\mathbf{e}_j) - f(\mathbf{x})}{\omega_{ij}^{[1]}\omega_{ij}^{[2]}} - \frac{g_i}{\omega_{ij}^{[2]}} - \frac{g_j}{\omega_{ij}^{[1]}}. \tag{32}$$

From a practical standpoint, Eq. (32) requires only a single additional function evaluation per entry, making the forward Hessian considerably cheaper than its central counterpart, at the expense of increased sensitivity to noise. To avoid redundant evaluations, all four low-level routines accept the $f(\mathbf{x})$ value as an explicit argument, and `fwhess` also takes the $\mathbf{g}$ vector at $\mathbf{x}$ (these quantities are computed automatically by `get_num_grad` and `get_num_hess`, so that the users need not supply them). In addition, `cdgrad` optionally returns the diagonal of the Hessian as a byproduct, without any extra function evaluations beyond those already required for the gradient itself. As noted in Sec. 2.1, only the lower-triangular Hessian entries are computed explicitly, since the gain from averaging h_{ij} and h_{ji} does not justify the extra function evaluations that the upper-triangular part would necessitate. Finally, fixation of variables is handled differently across the two levels of the module: the top-level routines accept an optional logical mask vector (`fixed`), while the low-level functions infer from the zero elements of the `fdconfig%fdoffsets` array whether a given derivative entry is to be evaluated or set to zero (the `fixed` array is also used in `mod_fdtune`).

3.2. Noise estimation (`get_noise_table`*)*

The `get_noise_table` function determines the function-noise parameters, required for the computation of the optimal finite-difference offsets of Sec. 2.2. This function returns an $n \times 2$ array (`noise_table`): the first column stores the noise-floor offset (ω_i^{noise}) for each variable, and the second one stores the respective noise estimate (f_i^{noise}). The procedure consists of two sequential phases carried out independently for each free variable x_i: (a) a **coarse scan** that detects an offset range where the function response is above the roundoff floor, as well as (b) a **fine scan** that samples the function change at various offsets and calculates the noise *via* Lagrange extrapolation to zero offset.[6]

Before entering the per-variable loop, the procedure evaluates the roundoff-level function change, employing the machine epsilon $\varepsilon^{\text{mach}}$ ($\approx 2.2 \times 10^{-16}$ for 64-bit real numbers[2]):

$$f^{\text{round}} = \varepsilon^{\text{mach}} \max(1, |f|), \tag{33}$$

where $f = f(\mathbf{x})$ denotes the reference function value (optionally taken from the argument list). For each variable x_i, the roundoff-level offset is

$$\omega_i^{\text{round}} = \varepsilon^{\text{mach}} \max(1, |x_i|), \tag{34}$$

delivering a minimum safe probing offset,

$$\omega_i^{\text{safe}} = \omega_i^{\text{round}} / \sqrt{\varepsilon^{\text{mach}}}. \tag{35}$$

At this scale, the function response is no longer completely buried in roundoff noise, yet the perturbation is still small enough to remain in a regime dominated by the noise floor rather than the analytic signal.

The purpose of the **coarse scan** is to find an offset ω_i at which $|f(\mathbf{x}+\omega_i\mathbf{e}_i)-f|$ exceeds the roundoff threshold by a factor of at least r^{round} (default value: $r^{\text{round}} = 5$), so that subsequent noise measurements are not dominated by floating-point cancellation. The iteration starts from the maximum offset $\omega_i^{\max}$ (user-defined) and executes at most `mnoise_scans` steps (default: 15). At each step, the function change is measured as

$$\Delta f_i = \max(|f(\mathbf{x} + \omega_i \mathbf{e}_i) - f|, f^{\text{round}}), \tag{36}$$

where the max guard prevents division by zero in the subsequent estimate of the new offset,

$$\omega_i^* = \max\left(f^{\text{round}} \omega_i / \Delta f_i, \omega_i^{\text{lb}}\right), \tag{37}$$

and ω_i^{lb} is a lower bound enforced on ω_i^* ($\omega_i^{\text{lb}} = \omega_i^{\text{safe}}$ in the first iteration and $\omega_i^{\text{lb}} = \omega_i^{\text{round}}$ in the subsequent ones). If $\Delta f_i \gg f^{\text{round}}$, the offset will be decreased, while it is acceptable for $\Delta f_i \approx f^{\text{round}}$. After that, the function change is probed at ω_i^* (hence denoted as Δf_i^*) and this new offset is accepted if

$$\Delta f_i^* \geq r^{\text{round}} f^{\text{round}} \qquad \text{or} \qquad |\omega_i^* - \omega_i| < r^{\text{off}} \omega_i, \tag{38}$$

where r^{off} is the relative offset tolerance (default: $r^{\text{off}} = 0.1$). The first condition checks that the signal is sufficiently above roundoff, whereas the second one watches whether the offset has become stable. If neither condition is satisfied, the iteration continues with $\omega_i \leftarrow \sqrt{\omega_i \omega_i^*}$. The geometric mean is chosen deliberately: since the search space spans many orders of magnitude, it contracts the interval much faster than the arithmetic mean would.

After the coarse scan has established a suitable starting offset, the **fine scan** samples the Δf_i values at geometrically increasing offsets and extrapolates the noise floor to zero *via* Lagrange interpolation. Let $\{(\omega_{i\#j}, \Delta f_{i\#j})\}_{j=1}^m$ be the sample points collected up to the mth fine-scan iteration. The unique Lagrange interpolating polynomial through these points, evaluated at $\omega_i = 0$, gives the extrapolated noise:

$$f_{i\#m}^{\text{extrap}} = \sum_{j=1}^{m} \Delta f_{i\#j} \prod_{\substack{k=1 \\ k\neq j}}^{m} \frac{\omega_{i\#k}}{\omega_{i\#j} - \omega_{i\#k}}. \tag{39}$$

In principle, high-degree Lagrange extrapolation can be sensitive to noise in the sample values; however, the fine scan converges in only a few steps in all test cases considered, so the polynomial degree remains low and no numerical instability is encountered in practice. The $\bar{f}_{i\#m}^{\text{noise}}$ estimate is then clamped to a reasonable range,

$$f_{i\#m}^{\text{noise}} = \mathcal{C}(|f_{i\#m}^{\text{extrap}}|, f^{\text{round}}, \Delta f_{i\#m}), \tag{40}$$

ensuring it never exceeds the measured signal nor falls below the roundoff floor. Afterwards, the offset is updated as $\omega_{i\#m+1} = 2r^{\text{noise}}\omega_{i\#m}$ and the associated $\Delta f_{i\#m+1}$ value is evaluated, wherein r^{noise} means the relative noise tolerance (default: $r^{\text{noise}} = 10$). The large multiplicative factor induces consecutive offsets spanning several decades, which enables robust extrapolation across a wide dynamic range. The scan terminates when either successive noise estimates agree or the signal has stabilized and further sampling is unlikely to refine the noise estimate. Formally,

$$\max(f_{i\#m-1}^{\text{noise}}, f_{i\#m}^{\text{noise}}) < r^{\text{noise}} \min(f_{i\#m-1}^{\text{noise}}, f_{i\#m}^{\text{noise}}) \tag{41}$$

or

$$\max(\Delta f_{i\#m-1}, \Delta f_{i\#m}) < r^{\text{change}} \min(\Delta f_{i\#m-1}, \Delta f_{i\#m}), \tag{42}$$

where r^{change} is the relative function-change tolerance (default: $r^{\text{change}} = 10$).

After the fine scan terminates at iteration m, the noise-floor offset ω_i^{noise} is set to $\omega_{i\#m-n-1}$ (or $\omega_{i\#m-n}$ if only a single fine-scan step was taken), where n is the number of iterations consumed by the coarse scan. This is the last offset at which the function response is still dominated by noise rather than the true signal. If any variable fails to converge within `mnoise_scans` coarse + fine steps, the (optional) global flag `cnvgd` is set to false, informing the caller that the noise estimates should be treated with caution.

3.3. Offset optimization (get_fdoffsets)

The get_fdoffsets function computes the optimal finite-difference offsets for all gradient and Hessian entries of the $f(\mathbf{x})$ function at a point $\mathbf{x}$. This procedure receives the f_i^{noise} noise levels, along with the prescribed offset ranges $[\omega_i^{\min}, \omega_i^{\max}]$, then returns the offsets in the 3D array fdoffsets following the layout of Sec. 2.1. These input data may be derived from get_noise_table: its second column provides the f_i^{noise} values, while its first column serves as the $\omega_i^{\min}$ estimates (note that the upper bounds $\omega_i^{\max}$ are always user-specified). The get_fdoffsets function accepts the following optional parameters: (a) a logical flag activating the approximate off-diagonal Hessian mode (approx; default false), (b) the relative offset tolerance (default 0.01), (c) the maximum number of offset iterations per entry (moff_scans; default 1), (d) the reference function value (evaluated internally if not supplied), and (e) a convergence flag (cnvgd), set to false when any offset is unable to converge within moff_scans iterations. In all test cases considered, moff_scans = 1 was sufficient to locate the optimum to within a factor of 10.

For all optimized offsets, an initial tight/loose "bracket" is formed from the input offset bounds. As the optimized direction is always associated with the last index, the bracket endpoints share the same form for all entry types:

$$\omega_{ij}^{[2]:\text{tight}} = \omega_j^{\text{tight}} = \sqrt{\omega_j^{\min}\,\omega_j^{\max}}, \tag{43}$$

$$\omega_{ij}^{[2]:\text{loose}} = \omega_j^{\text{loose}} = \sqrt{\omega_j^{\text{tight}}\,\omega_j^{\max}}. \tag{44}$$

These expressions serve as starting points for the **offset scan**, the iterative workflow behind the offset optimizations of the gradient and Hessian components [in analogy with Eqs. (12)–(14), entry-specific indices are suppressed in this paragraph]. The optimum candidate, ω^{opt}, is computed from Eqs. (4), (6), and (10), using the truncation coefficients of Eqs. (15), (19), and (23), respectively [in the approx mode, the gradient-only estimate of Eq. (27) is adopted]. The scan terminates if the candidate and the tight endpoint agree to within a factor of r^{off}:

$$\min(\omega^{\text{opt}}, \omega^{\text{tight}}) > r^{\text{off}} \max(\omega^{\text{opt}}, \omega^{\text{tight}}). \tag{45}$$

Otherwise, the derivative is re-evaluated at ω^{opt}, the farther bracket endpoint is replaced, the bracket is re-sorted to $\omega^{\text{tight}} \leq \omega^{\text{loose}}$, and the scan proceeds until Eq. (45) is met or moff_scans iterations are exhausted.

In the **gradient pass**, the offset scan is applied to each free variable x_i. The two gradient estimates, g_i^{tight} and g_i^{loose}, are evaluated at the bracket endpoints by calling cdgrad or fwgrad, and the truncation coefficient is derived from Eqs. (16)–(18) by setting $q = 1$. After convergence, ω_i^{opt} and t_i^{raw} are stored for subsequent use. In the case of forward differences ($p = 1$), the **Hessian pass** is a direct extension of the gradient pass. This is realized by choosing $\omega_{ij}^{[1]:\text{opt}} = \omega_i^{\text{opt}}$ and $\omega_{ij}^{[2]:\text{opt}} = \omega_j^{\text{opt}}$ for all lower-triangular entries, as required by the use of the cached gradient values in Eq. (32). Accordingly, no offset optimization is performed for the forward-difference Hessian (a joint optimization of the individual gradient and Hessian errors, though possible, is not implemented).

For central differences, all lower-triangular entries are processed by the Hessian pass, whose execution depends on both the entry type (diagonal or off-diagonal) and the value of the approx flag. If approx is false, a full offset scan is run with up to moff_scans iterations. During these scans, the primary offset is kept fixed at $\omega_{ij}^{[1]:\text{opt}} = \omega_i^{\text{opt}}$ for each Hessian entry, though this is only relevant to off-diagonal components [see Eq. (7)]. For a h_{ii} entry, the bracket is initialized from the cached $(h_{ii}^{\text{loose}}, h_{ii}^{\text{tight}})$ couple, returned by cdgrad as a byproduct of the gradient pass. After that, the offset scan proceeds with Eqs. (6) and (19)–(22), subject to $q = 2$. For an off-diagonal h_{ij} entry, the $\omega_{ij}^{[2]}$ offset is refined through Eqs. (7) and (23)–(26) by setting $q = 1$, where $(h_{ij}^{\text{loose}}, h_{ij}^{\text{tight}})$ comes from two cdhess calls. When approx is true, the offset scan is limited to a single iteration for all entries. For diagonal entries, the cached $(h_{ii}^{\text{loose}}, h_{ii}^{\text{tight}})$ pairs are still used as in the previous case, but without iteration. In contrast, the truncation coefficients of the off-diagonal entries are estimated using the gradient-only guess of Eq. (27).

4. Conclusions

The FDPACK library offers a highly efficient, self-contained implementation for adaptive offset selection in numerical gradient and Hessian computations. The implemented procedures cover the practically most relevant cases: (a) noise estimation along each coordinate direction and (b) offset optimization for all gradient and lower-triangular Hessian entries coming from forward/central difference formulas. Several extensions are possible in future work, including support for higher-order derivatives, additional stencils, and even more sophisticated noise models. Information on installation and testing is provided in files doc/INSTALL.md and doc/TESTS.md, respectively.

References

[1] Fornberg, B. Generation of finite difference formulas on arbitrarily spaced grids. *Math. Comp.* **51**, 699–706 (1988). URL.

[2] Higham, N. J. *Accuracy and Stability of Numerical Algorithms* (Society for Industrial and Applied Mathematics, Philadelphia, 2002), 2nd edn. URL.

[3] Stepleman, R. S. & Winarsky, N. D. Adaptive numerical differentiation. *Math. Comp.* **33**, 1257–1264 (1979). URL.

[4] Richardson, L. F. The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam. *Phil. Trans. R. Soc. Lond. Ser. A* **210**, 307–357 (1911). URL.

[5] Anderson, E. *et al. LAPACK Users' Guide* (Society for Industrial and Applied Mathematics, Philadelphia, 1999), 3rd edn. URL.

[6] Abramowitz, M. & Stegun, I. A. *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables* (National Bureau of Standards, Washington D.C., 1964).